SECTION 1

Learning to optimize rewards

- **Agent–environment loop:** An agent observes, acts, and collects rewards in domains from robotics and games to thermostats and trading; the objective is maximize discounted future return, not supervised labels.
- **Policies:** The policy maps states (or histories) to actions and may be deterministic or stochastic; naive policy search over parameters or genetic algorithms explode in dimension.
- **Deep RL context:** Combining deep networks with RL enabled landmark results (e.g., Atari from pixels, Go), motivating the algorithms that follow.

SECTION 2

Policy gradients and CartPole

- **Credit assignment:** Sparse delayed rewards make it unclear which actions helped; discounted returns and normalized advantages score actions after full episodes.
- **REINFORCE-style training:** Record gradients that would increase chosen actions, then scale by advantage and average across episodes before applying Adam updates on a small neural policy.
- **Exploration:** Stochastic action sampling balances explore vs exploit; prior knowledge (reward shaping, imitation) can dramatically shorten training.

SECTION 3

OpenAI Gym and neural policies

- **Gym API:** Environments expose `reset()`, `step()` returning observation, reward, done, info, plus discrete or continuous action spaces—CartPole is the introductory benchmark.
- **Neural policy:** A compact MLP outputs action probabilities; the chapter contrasts a hand-coded tilt heuristic with a learned policy that balances the pole far longer.
- **Simulation first:** Gym (and physics simulators) let agents fail safely and parallelize experience before real-world deployment.

SECTION 4

Markov decision processes and tabular methods

- **MDPs:** States, actions, transition probabilities T(s,a,s′), rewards R, and discount γ underpin Bellman optimality and value / Q-value iteration on toy graphs.
- **Temporal difference learning:** With unknown dynamics, TD updates bootstrap from observed transitions using learning rate α instead of full model knowledge.
- **Q-learning:** Off-policy updates estimate Q(s,a) toward r + γ max■′ Q(s′,a′); ε-greedy or curiosity-weighted exploration balances coverage and exploitation.

SECTION 5

Deep Q-learning

- **Function approximation:** DQNs regress Qθ(s,a) for all actions from one state input, essential when tabular Q is impossible (e.g., visual Atari states).
- **Replay and targets:** A replay buffer decorrelates mini-batches; target Q-values use r + γ max Q(s′) with gradient steps on MSE or Huber loss.
- **Instability:** Catastrophic forgetting, correlated samples, and sensitivity to architecture and seed make RL hard to tune—monitor returns, not loss alone.

SECTION 6

DQN variants and TF-Agents

- **Stabilizers:** A target network, Double DQN (online picks action, target evaluates), prioritized replay, and Dueling heads (value + advantage) address overestimation and sample efficiency; Rainbow combines several tricks.
- **TF-Agents:** High-level APIs wrap Breakout and other Gym/Atari envs with replay buffers, drivers, and DQN training loops on pixels after lengthy training.
- **Operational lesson:** Libraries bundle battle-tested hyperparameters (large buffers, slow ε decay, infrequent target sync) needed for pixel RL.

SECTION 7

Survey of further algorithms

- **Actor–critic family:** Policies learn with a critic estimating values or advantages; variants include A3C/A2C, entropy-maximizing SAC, and clipped PPO used in large-scale game agents.
- **Exploration research:** Curiosity-driven intrinsic rewards encourage visiting unpredictable states when extrinsic rewards are sparse.
- **Takeaway:** Policy gradients and value-based deep RL complement each other; sample efficiency and stability remain active research fronts.